Evolution 0 · Lecture

The Human Is Index Zero

Layers of AI · the AI → Agentic AI progression · what makes a system an agent · the Degradation Pattern · the covenant and the five gates

SURVEY **BUILD** **GOVERN** **DEFEND**

CIDM 6333 · Agentic Systems Design and Development
Dr. Jeffry Babb · Paul and Virginia Engler College of Business · West Texas A&M University

Sixty minutes, five moves

0:00–0:03

The run of show for tonight's lecture

1	0:00	**Framing**	The question behind the course; Tree Hollow; what 'answerable' means
2	0:10	**Two maps**	Layers of AI; AI & ML → Agentic AI; the framework board by week
3	0:25	**What is an agent?**	Russell & Norvig; the loop and its halt; nine components; a worked Design Analysis
4	0:38	**Why governance**	The Degradation Pattern; the Zero Doctrine; the covenant; five gates, two lanes; command doctrine
5	0:52	**The instrument and the brief**	The judgment loop; Evolution 0: readings, first Design Analysis, the pitch

The question behind the course

0:03–0:10

Why are agentic systems designed the way they are — and can you build one you can answer for?

> *"A system can produce an answer without being able to answer for it."*
>
> Sprint and Drift, "Tree Hollow" — this week's essay

Survey

Why was each system built that way, and what did it cost? Historical order, one component model.

Build

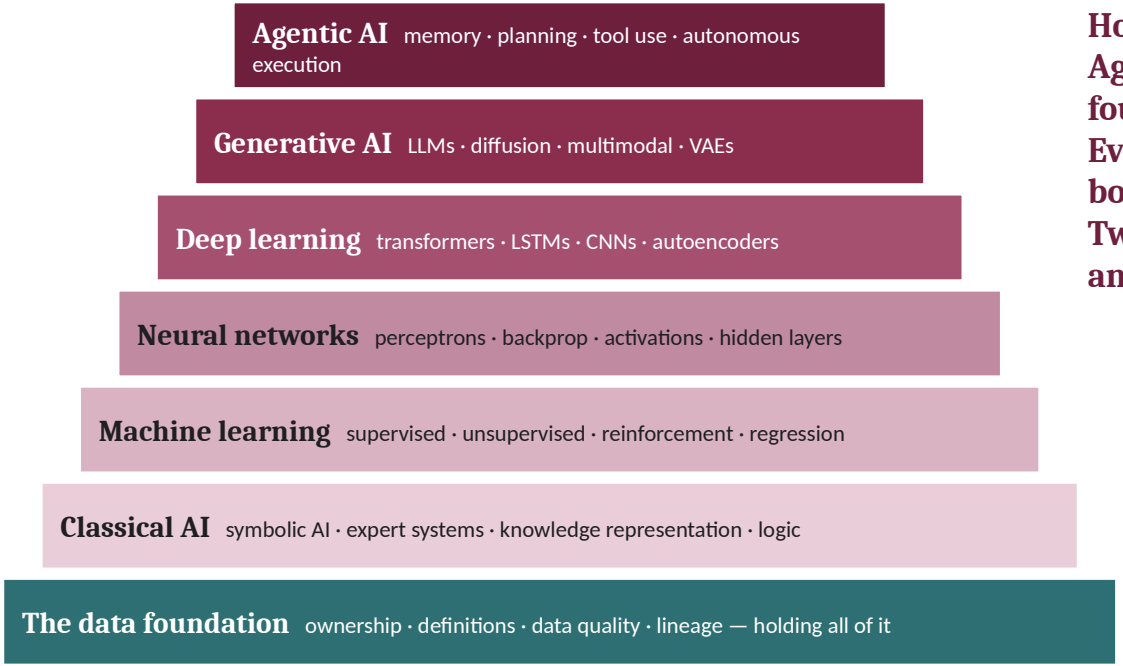
One system, one layer a week, every layer flown against a claim that a single observation could disprove.

Answerability
A model has no persistent state, no ground truth, no accountability. The engineer is the one who answers.

Map one: the Layers of AI

0:10–0:15

Seven layers; each stands on the one below; none replaces it



How to read it
Agentic AI is thin and new; the foundation is old and cracked.
Every Design Analysis starts at the bottom: what model, weights, data?
Two passes: what it claims (top-down) and what it stands on (bottom-up).

...and the one everybody forgets

The data foundation is unglamorous, and it is where the cost lands

Ownership

Who may the agent read, write, and act on behalf of? Whose data trains the model it stands on?

Definitions

What does 'customer', 'done', 'passed' mean to the system? An agent inherits every ambiguity in the schema.

Quality

Retrieval returns what is stored. A memory store with one false fact propagates it as truth (E7's exercise).

Lineage

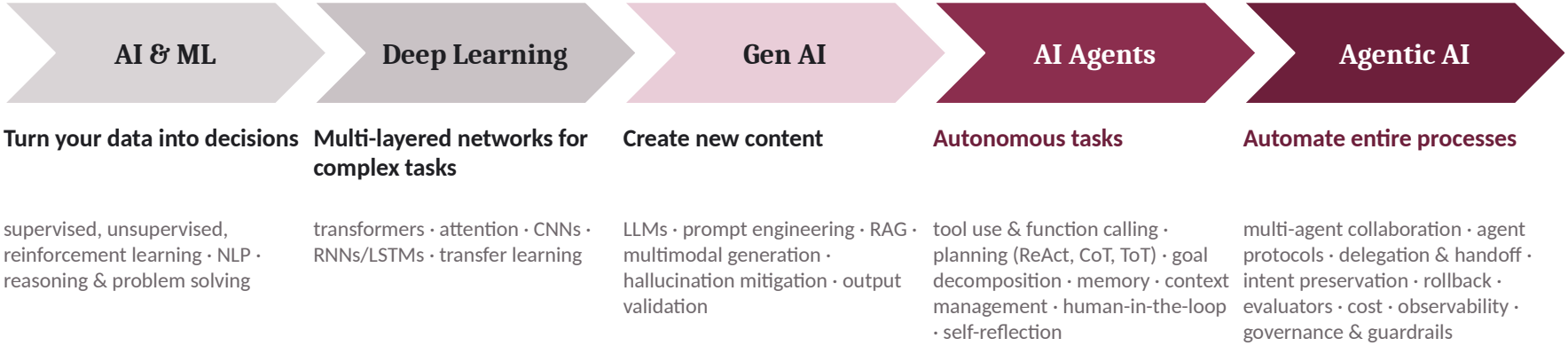
Where did this answer come from? Without provenance a summary becomes a 'fact' — the summary-laundering failure (E9).

The Newtonian-motion argument: the cost of generated code is deferred, never erased.

Map two: from AI & ML to Agentic AI

0:15–0:19

The five-stage progression from Agentic AI: A Complete Framework

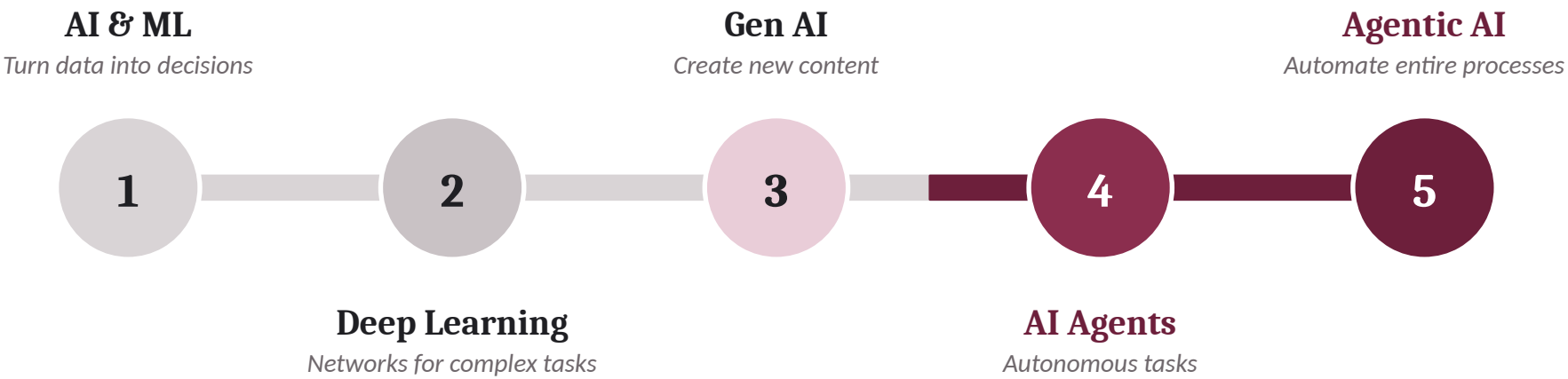


The course lives in the last two stages. E1–E7 build the AI Agents column; E8–E12 build the Agentic AI column. The unit of work goes from decision, to artifact, to task, to process.

The road map

0:19

Five stops from data to automated processes — you are building toward the last two



This course lives here. E1–E7 build stop 4 on your system; E8–E12 build stop 5.

Adapted from Agentic AI: A Complete Framework.

The framework board, read as rings

0:19–0:22

Foundation → creation → core systems → action → full automation

1	**Foundation**	Data, models, weights, compute. Local or hosted. The layer everything else stands on.
2	**Creation**	Generative capability: LLMs, prompting, RAG, output validation, hallucination mitigation.
3	**Core systems**	Reasoning & planning · tool use & function calling · memory systems · context management.
4	**Action**	Autonomous execution · dynamic tooling · self-reflection & error recovery · human-in-the-loop.
5	**Full automation**	Multi-agent collaboration · protocols · delegation · intent preservation · rollback · evaluators · cost · observability · governance.

Two ways to read the board

Rings: how far capability extends from the foundation. Columns: when each capability arrived.

Claims ring five, runs on ring two: the most common thing marketed as 'agentic'.

Mark every box you can name a system for; the blanks are your reading list.

Every box on the board is assigned to a week

0:22–0:25

Read the map for the semester before you fill it in

Week	Layer built	Framework-board boxes	Survey systems
E0	Toolkit; pitch; first Design Analysis	The whole progression; Governance & Future	Chat assistant vs. Claude Code; PEAS
E1	Governed agent loop with a halt; PRD	Reasoning & problem solving; planning (ReAct, CoT, ToT); autonomous execution	ReAct; AutoGPT; BabyAGI
E2	Named pattern; framework; ADR-1	Frameworks & runtimes; goal decomposition	LangGraph; Pydantic AI; AutoGen; CrewAI; vendor SDKs
E3 ★	Schema-validated tools; permission gating	Tool use & function calling; output validation; dynamic tooling	Toolformer; function calling; computer use; hooks
E4–E5	MCP server; hardened; instruction files	Tool orchestration; agent protocols; governance, safety & guardrails	Plugins → MCP; A2A; AGENTS.md; harness engineering
E6–E7 ★	Context budget; handoff; memory; ADR-2	Context management; memory systems; memory governance & retention; RAG	Claude Code / Codex context; RAG; Generative Agents; MemGPT; Reflexion
E8–E9	Implementer–reviewer pair; adversarial validation; locks; routing	Multi-agent collaboration; delegation & handoff; cost & resource mgmt; marketplaces & contracts	FIPA/BDI; CAMEL; MetaGPT; AutoGen; Cemri et al.
E10–E11	Eval suite; RED witness; tracing; red-team card	Feedback loops & evaluators; self-reflection & error recovery; observability & tracing; risk	SWE-bench; GAIA; LLM-as-judge; system cards
E12	Served, containerized, released, rollback	Task scheduling; rollback; long-term autonomy; self-improving agents	Background agents; Copilot coding agent

★ *local open-weight model required. Full mapping: the Course Topic Guide, §4.1 traceability matrix.*

What a language model adds — and what it does not

0:25–0:27

The stack above the model exists to supply what the model lacks

What it adds

A reasoner that generalizes from text.

It reads an unfamiliar error, proposes a plan, writes the call, and explains itself. The capability classical agents never had.

No persistent state

Every session starts empty. Context, memory, handoffs, and ledgers (E6–E7) exist to supply it.

No ground truth

It has no environment of its own. Tools, retrieval, and evaluators (E3–E5, E10–E11) exist to supply it.

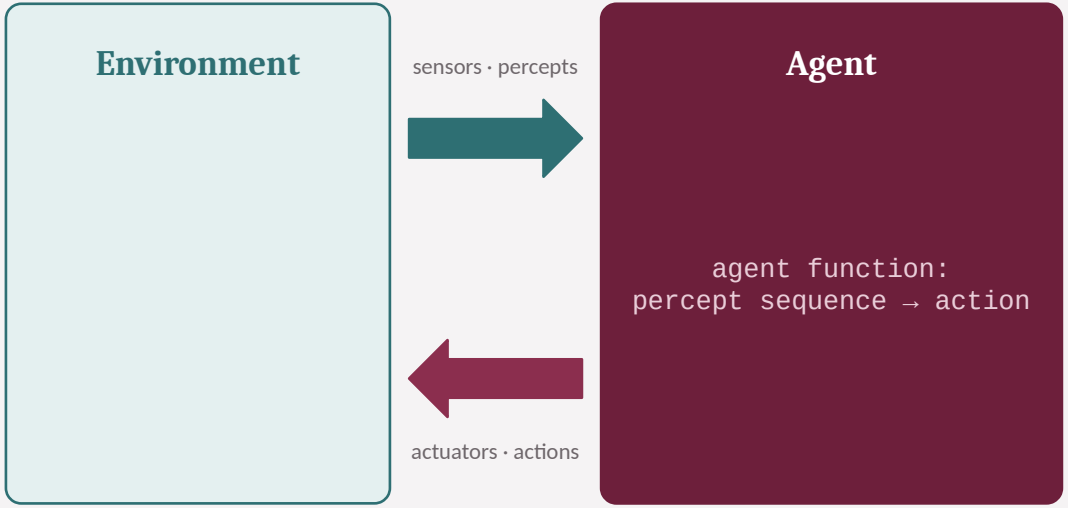
No accountability

It cannot answer for anything. Governance (E0, E5, E9, E11) supplies it and cannot be automated.

What is an agent? The definition that predates the LLM

0:27–0:30

Russell & Norvig: perceives through sensors, acts through actuators



P — **Performance measure**
What counts as success — the thing 2023's loops never wrote down

E — **Environment**
Repo, shell, browser, API, a user; observable or partial; deterministic or not

A — **Actuators**
Tool calls, file writes, messages, commits — the side-effect surface

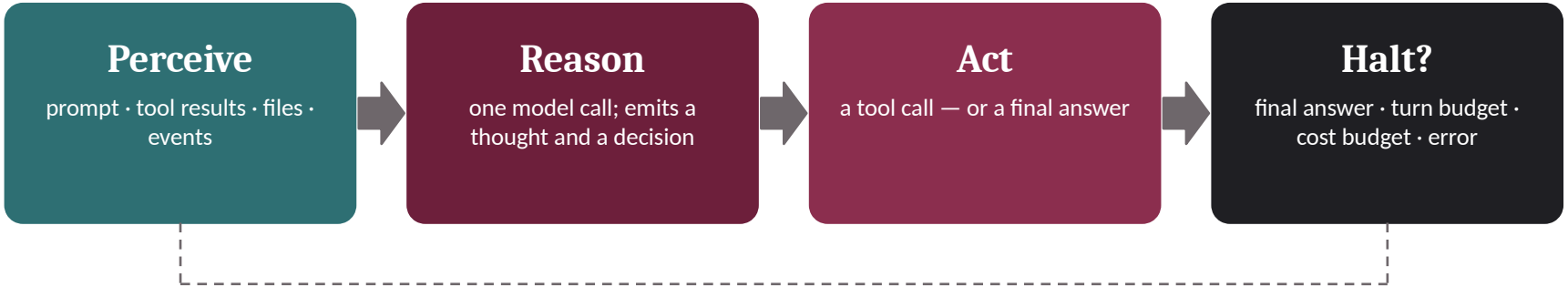
S — **Sensors**
Prompts, tool results, screenshots, file contents, events

Agent types: simple reflex · model-based · goal-based · utility-based · learning. BDI and FIPA return in E8.

The LLM agent loop

0:30–0:33

The classical definition with a model in the reasoner slot — and a halt condition



ReAct (Yao et al., 2022): the canonical form

Thought → Action → Observation, until the model emits a final answer. The atom every later week decorates.

The halt condition is the part 2023 forgot

AutoGPT and BabyAGI (spring 2023): goals, no grounding, no budget, no evaluator. A loop without a halt is a bill. Sortie 1 is 'a governed loop that halts.'

The nine components of a Design Analysis

0:33–0:36

The question to ask of every system — and the failure that appears when the component is missing

1 Foundation

What model, weights, data? Local or hosted?

× *Cost and capability assumed, not measured*

2 Perception

What does it see: text, tool results, screens, files, events?

× *Untrusted input treated as instruction (injection)*

3 Planning & reasoning

ReAct? CoT/ToT? Decomposition? Fixed workflow or open loop?

× *Spirals; no halt; plans that never replan*

4 Tools & orchestration

How are tools declared, called, permissioned, recovered?

× *Unschema'd calls; writes without confirmation*

5 Memory & context

What is in the window, compacted, persisted; who owns retention?

× *Context exhaustion; stale facts recalled as truth*

6 Coordination

Single, controller–worker, crew, graph, protocol (MCP/A2A)?

× *Summary laundering; credit inferred, not earned*

7 Evaluation & feedback

What checks it, what it optimizes, how failure is detected?

× *Gaming the gate; judge self-preference*

8 Governance & human

Where is the human: index zero, in/on the loop, absent? What is attested?

× *Governance theater; rubber-stamp attestation*

9 Runtime & operations

Scheduled, observed, rolled back, billed — how?

× *No rollback; unmeasured cost and latency*

A worked Design Analysis: chat assistant vs. agent harness

0:36–0:40

The E0 exercise, done on a slide — what makes the second one an agent?

Component	Chat assistant (2022–)	Claude Code / Codex CLI (2025)
Foundation	Hosted frontier model	Hosted frontier models; model tiers per task; no local option
Perception	The user's text; pasted content; maybe a file upload	Repo files, shell output, tool results, CLAUDE.md, hook output
Planning	One turn; the human is the loop	Open ReAct loop; plan mode; task decomposition; subagents
Tools	None (or a closed set the vendor runs)	Built-in read/edit/bash + MCP; permission prompts; hooks
Memory & context	The window, then gone; optional vendor memory	Window + compaction; CLAUDE.md as instruction file; no long-term memory
Coordination	Single	Main session + subagents; Agent tool; background tasks
Evaluation	The human reads the answer	Tests it runs; the human; nothing built in
Governance & human	Human on every turn by construction	Human in the loop per tool call; attestation is implicit
Runtime	A web page	Terminal; background tasks; cloud sessions

Why, and what it cost

The harness owns the loop: tools, permissions, hooks, compaction, subagents ship as features.

The cost: governance is per-call and implicit; nothing records what was attested. gzkit fills that gap (component 8).

The chat assistant is a workflow with the human as the loop.

Two distinctions to carry out of tonight

0:40–0:42

Workflows versus agents; what the pre-LLM era carried forward and what it dropped

Workflows versus agents (Anthropic, 2024)

Workflow: the model runs along code paths you wrote.

Agent: the model directs its own control flow.

Most production 'agents' are workflows, and should be. E2 refactors your loop into a named pattern.

What carried forward — and what was dropped, then rediscovered

Carried forward
PEAS and the agent function
Planning: STRIPS → goal decomposition
Coordination: FIPA → A2A
Dropped in 2023, rediscovered
Explicit state → context engineering, ledgers
Halt conditions → governed loops
Evaluation → evals as gates
Reward hacking → gaming the gate (failure mode 9)

Why governance: the Degradation Pattern

0:42–0:46

“You didn't lose control in one dramatic moment.”

Week 1

You prompt carefully, review every line, and understand what is built.

Week 2

You skim diffs and trust the green checkmarks.

Week 3

You batch approvals; “looks good” becomes reflexive.

Week 4

You approve PRs you did not read, for code you do not understand.

Three names for the same failure

Cognitive surrender
the judgment is delegated, not the task

Mediated observation
you see the summary, never the artifact

Governance theater
every check passes and nobody verified anything

The Zero Doctrine and the covenant

0:46–0:49

The human is index zero — and the obligation runs both ways

[0]

First in priority.
Your intent drives every decision.

Final in authority.
You sign off, not the model.

Cannot be automated away.
Accountability is a property of a person, not a pipeline.

How 'zero' evolved: zero overhead → zero drift → index zero.

The covenant: five clauses, two parties bound

1. The human defines intent (PRD, ADRs, acceptance criteria).
2. The agent operates within constraints and flags violations.
3. Verification is mutual: tests, checks, evidence.
4. The human attests completion after observing artifacts.
5. Artifacts survive sessions.

Two sides: gates constrain agents, and humans must engage. Gates alone produce theater. "Specification without governance drifts; governance without methodology is theater; methodology without specification is arbitrary."

Five gates, two lanes, and the command doctrine

0:49–0:52

Ceremony must earn its place; attestation cannot be automated

Gate	What it proves	Lite	Heavy
1 ADR	Intent and trade-offs recorded before implementation	●	●
2 TDD	Tests derived from the spec pass; RED witnessed first	●	●
3 Docs	Documentation describes actual behavior		●
4 BDD	External contracts verified by acceptance scenarios		●
5 Human	A human observed the artifacts and attested	●*	●

*Evolutions run Lite (gates 1–2). The Final System closes out Heavy (1–5). * Human attestation is universal; no self-close path. Closed vocabulary: Completed · Completed — Partial · Dropped.*

Command doctrine: the Four P's

Degani & Wiener: philosophy → policies → procedures → practices. The cockpit is the model.

- Accountability is non-transferable.
- Authority must be instrumented, not asserted.
- The model is a crew resource, not a crew member.
- Evidence is artifacts, not narration.

Where a covenant sits among the governance frameworks

0:52–0:54

Policy frameworks say what must be true; a covenant says who does what on Wednesday

Policy and standards

`NIST AI RMF · EU AI Act · ISO/IEC 42001`

Map, measure, manage, govern. Necessary, and silent on what one engineer and one agent do in one session.

Frontier-lab practice

`Safety frameworks · system cards`

What was tested, what was found. E11 reads one as a Design Analysis. They govern the model, not your use of it.

The accountability gap

`Raji et al.; “Auditing Work”`

Audits without access, standards without enforcement, attestations nobody observed. A covenant is one engineer's answer.

Where it fits: below policy, inside the session, at the level of the artifact. It cannot make a model safe, certify an organization, or substitute for law. It makes one build answerable.

The instrument: the judgment loop

0:54–0:56

From CIDM 6330 — five questions that turn a description into a defense

1. **Decision** What did they (or you) actually decide? One sentence.
2. **Characteristics privileged** What quality did the decision buy: speed, safety, cost, portability, control?
3. **Cost** What was given up to get it? Every design choice has a price.
4. **Failure mode** What breaks first when the assumption behind the decision stops holding?
5. **Preservation mechanism** What keeps the intent alive after the person who made the decision has left the session?

Design Analysis format

~150 words · weekly · design-analyses/<system>.md · one row in LEDGER.md

1. Sources read
2. Nine components, one line each
3. Why it was built this way, and what it cost
4. Judgment loop (five items)

Defended in consultation.

This week: Evolution 0

0:56–1:00

Due on the date in the course calendar. The mechanics are in the orientation deck and WTClass

Read

- Sprint and Drift, “Tree Hollow”
- gzkit why.md (the Degradation Pattern) and the glossary
- The two framework images: Layers of AI; Agentic AI: A Complete Framework
- Optional: Russell & Norvig ch. 2; Anthropic, “Building Effective Agents”

Write

- Design Analysis #1: chat assistant vs. agent harness, nine components each
- Your intended system specified by PEAS, mapped to components
- The framework board marked; the blanks are your reading list
- One-page system pitch: route, harness, candidate framework, risk

Stand up

- Toolkit: uv, Python 3.14, gzkit, your harness, Ollama (install clinic in lab)
- Course repository in the GitHub org; AI_LOG.md started from the first prompt
- Skills inventory
- Consultation: your system idea, tested against “can it take all twelve layers?”

The systems you will study were designed by people who had to answer for them.

Build one you can answer for.